

Deep Learning

Object Detection and Tracking

Outline

Introduction

Evolution of Object Detection Technologies

Deep Learning Era

- R-CNN

- Single Shot Detectors

YOLO (You Only Look Once) Family

- YOLOv1 to YOLOv3 (2016–2018)

- YOLOv4 and YOLOv5 (2020)

- YOLOv6, YOLOv7, YOLOv8 (2022–2024)

Outline

Object Tracking Technologies

- ByteTrack

- DeepSORT

SAM (Segment Anything Model)

- SAM v1 (2023)

- SAM v2 (2024–2025)

YOLO v11/v12 All-In-One

Code Walking (YOLO, YOLO + Tracking, SAM, YOLO Segmentation)

Introduction

Object Detection

Identifying and **localizing** objects in Images.

Object Tracking

Maintaining object Identity across frames once detected.

Introduction

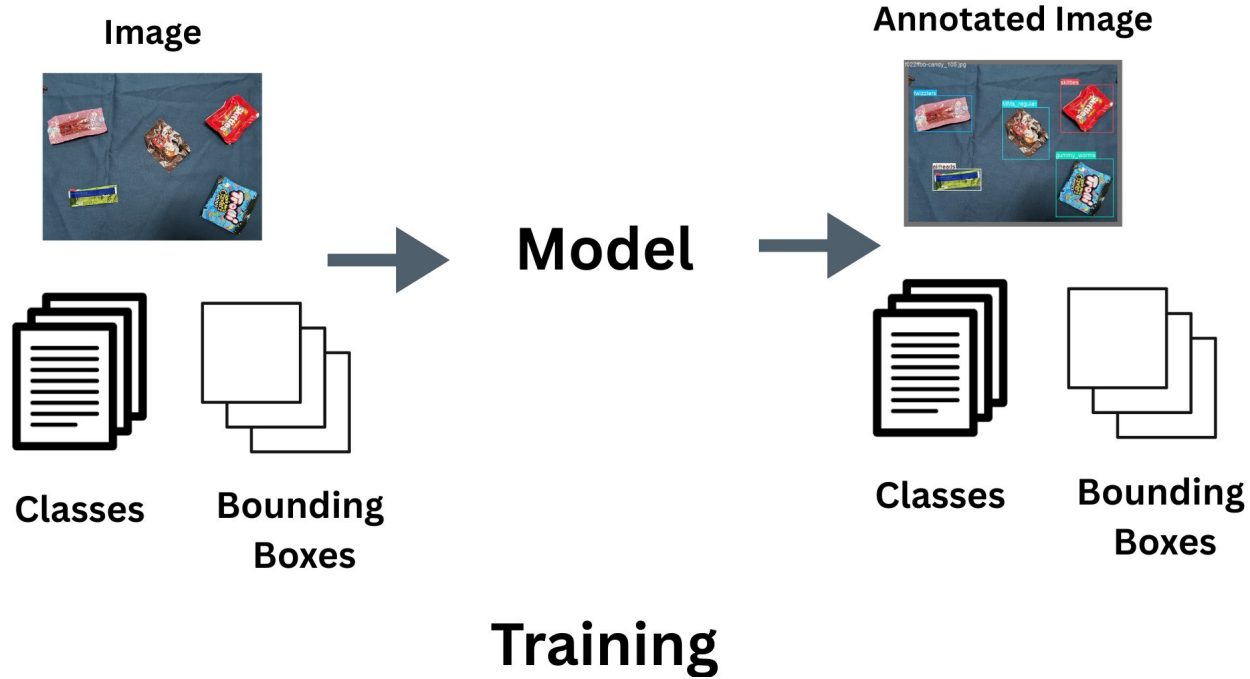
Bounding Box (BBx) Detection

Object Localization by enclosing targets within **Rectangular Boxes**.

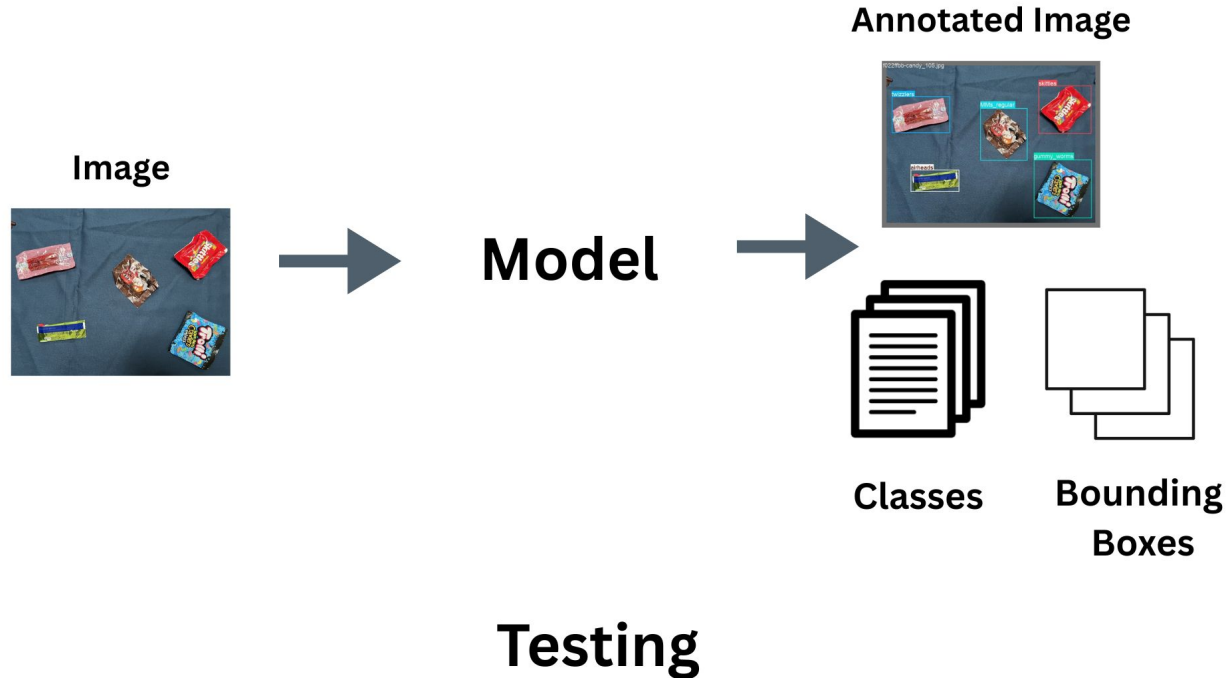
Segmentation

Pixel-Level Delineation of object boundaries for fine-grained understanding.

Introduction



Introduction



Evolution of Object Detection Technologies

Approaches:

HOG (Histogram of Oriented Gradients)

SIFT (Scale-Invariant Feature Transform)

Haar Cascades (Viola–Jones)

Limitations:

Handcrafted features → Poor scalability

Sensitive to illumination, occlusion, and perspective variations

Deep Learning Era

R-CNN

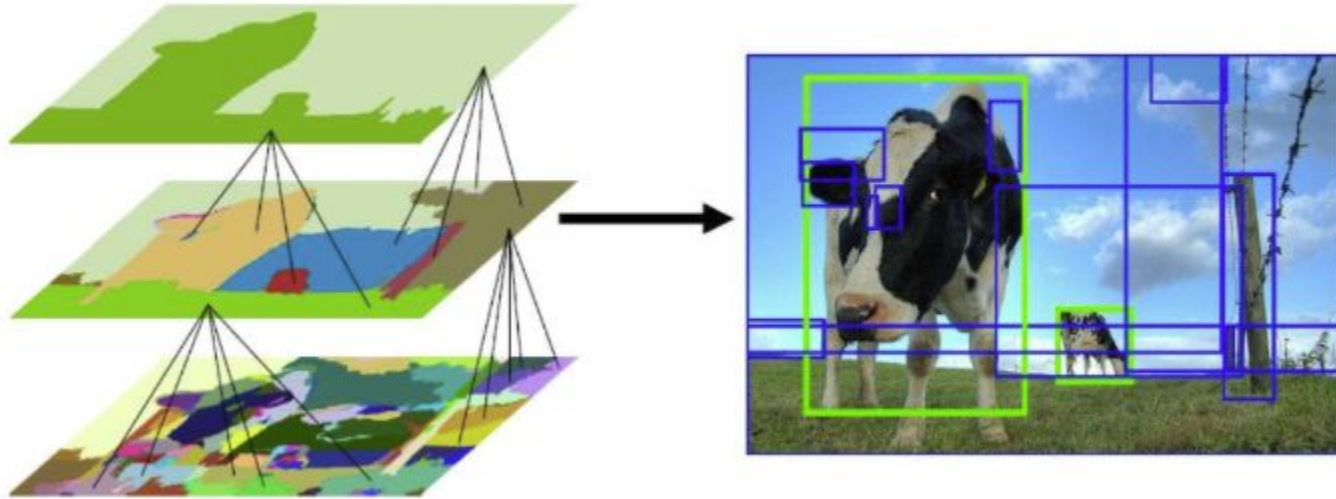
Region-Based Detectors

Proposal Region of Interest based on Feature Descriptor

Selective Search Generate Segment + Hierarchical Grouping

Classify that Region to a Class + Bounding Box Generation

Deep Learning Era



Deep Learning Era

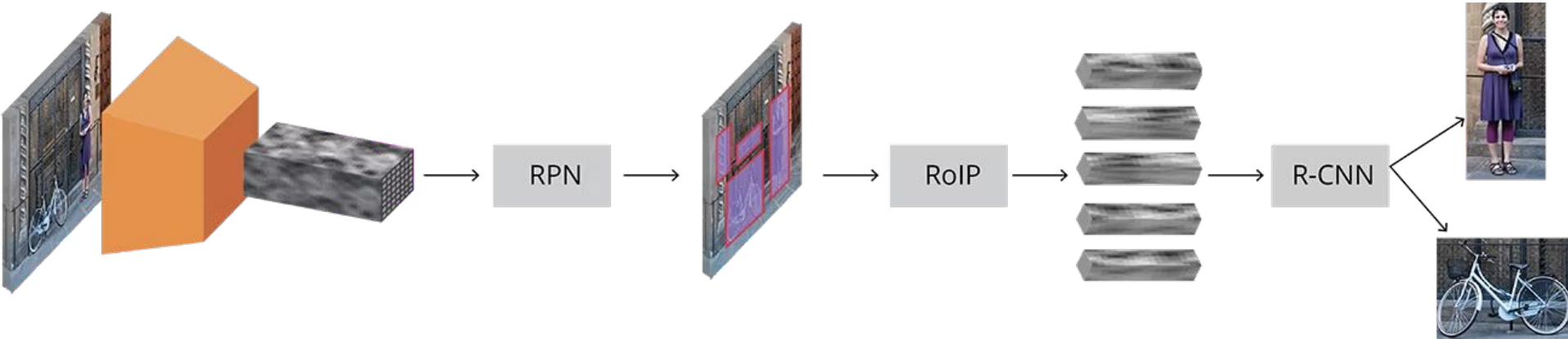
Fast R-CNN

Region Proposal Network

Base CNN + RPN + RP + ROI + R-CNN

RPN uses **Non-Maximum Suppression** Based on IOU

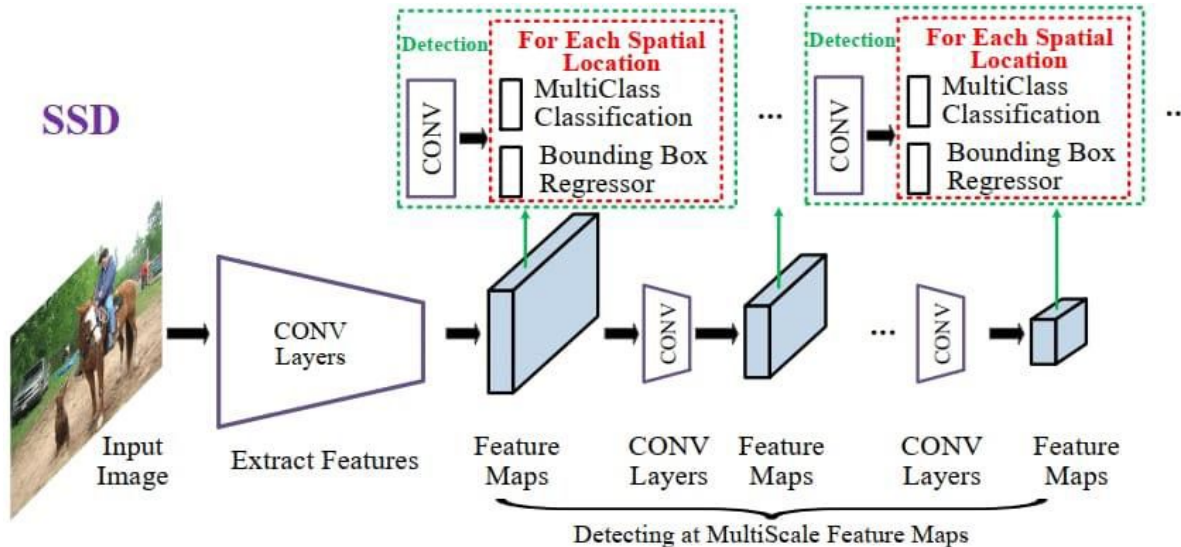
Deep Learning Era



Deep Learning Era

SSD (Single Shot MultiBox Detector, 2016)

Performs **Object Detection** in a **Single Forward Pass** using multiple feature maps from different layers of the network hence the name **Single Shot**.



Deep Learning Era

YOLO (You Only Look Once, 2016)

Backbone Neck Head

Treats detection as a

Single Regression Task

Dividing the image into an $S \times S$ grid where each cell directly predicts **Bounding Boxes**

Objectness Scores

Class Probabilities in one Unified, End-to-End Pipeline

YOLOv1 to YOLOv3 (2016 - 2018)

YOLO

GoogLeNet inspired CNN (24 Conv + 2 FC Layers)

DarkNet-19 as Backbone (19 conv + 5 MP layers)

Multi-Scale Training → Robust to different Image Sizes

Darknet-53 (53 conv layers with **Residual** Connections)

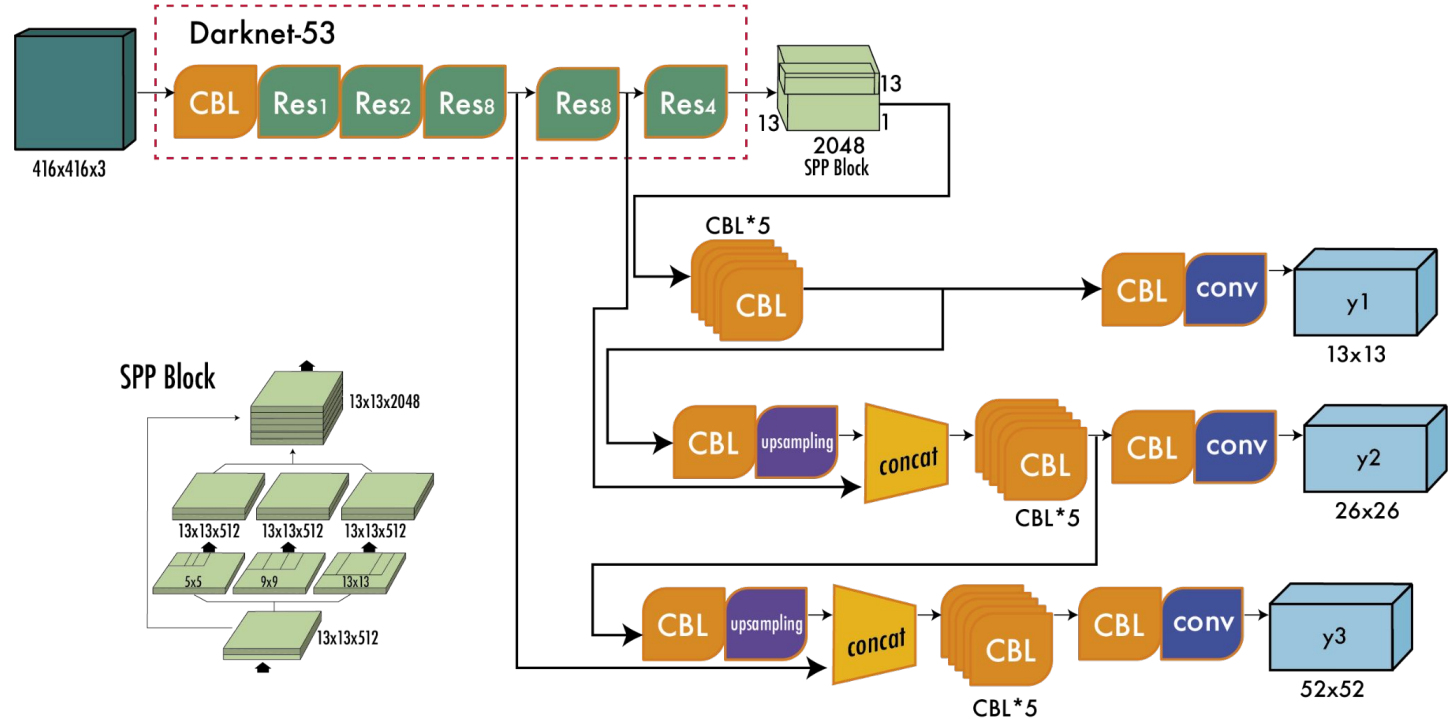
Logistic Classifiers instead of **Softmax** → allows Multi-Label Prediction

SPSS Block Capture Features at Multiple Receptive Field Sizes

Mish Activation $\Rightarrow x \times \tanh(\ln(1 + e^x))$

YOLOv1 to YOLOv3 (2016 - 2018)

v3



YOLOv4 to YOLOv7 (2020–2021)

YOLO

CSPDarknet Backbone **Cross Stage Partial Connections**

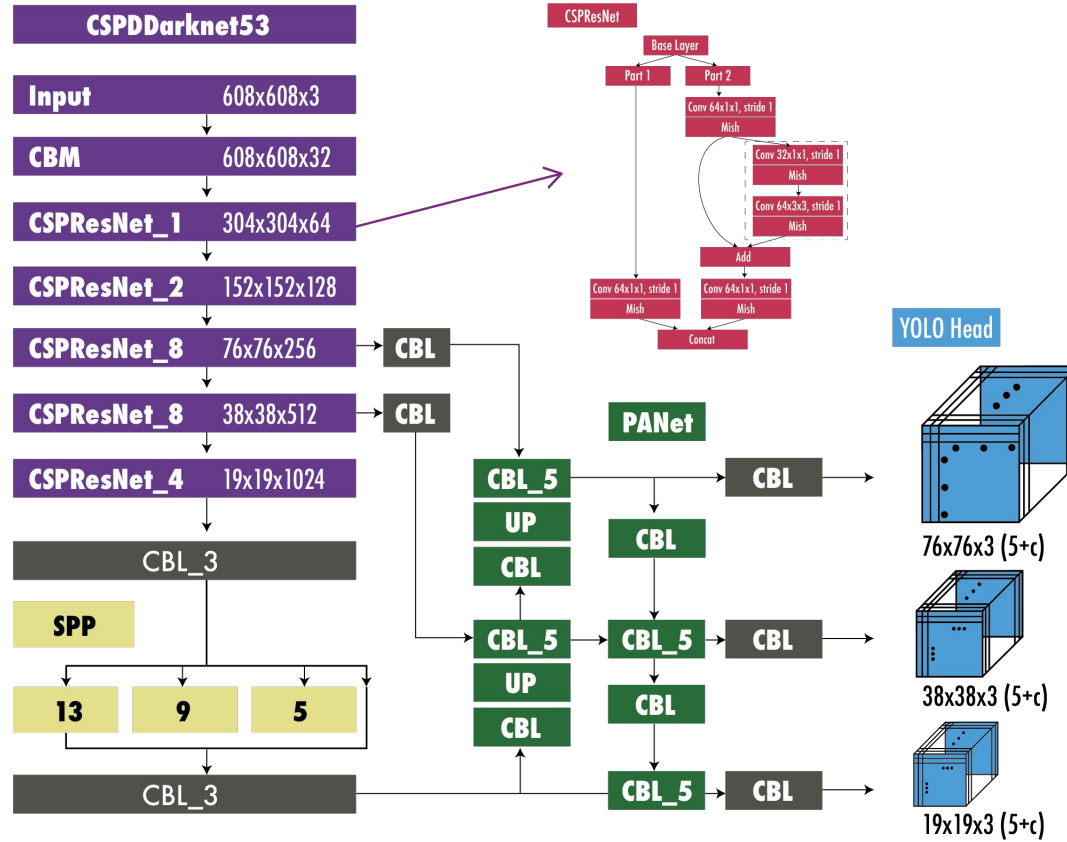
PANet Path Aggregation **Combines Bottom-Up and Top-Down Pathways**

Advanced Augmentations (**Mosaic**, **CutMix**)

Complete IoU/ Distance IoU Loss for better Bounding Box Regression

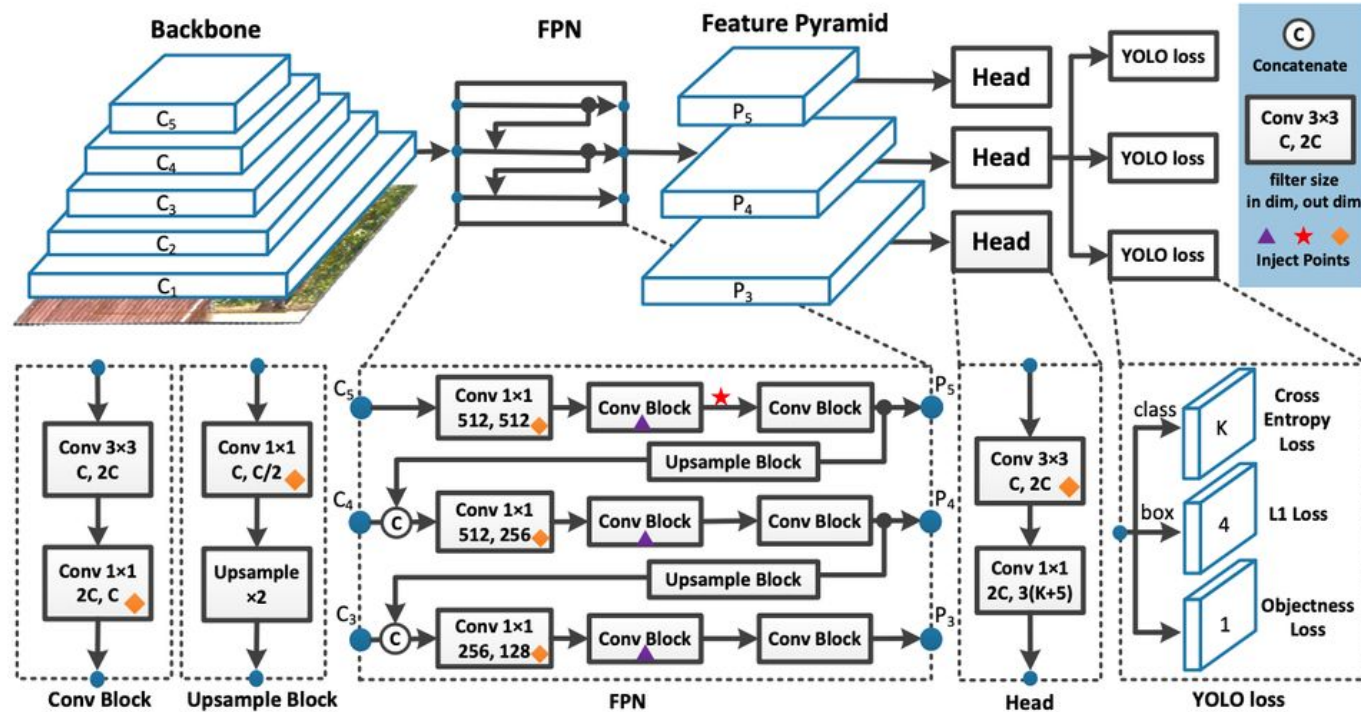
YOLOv1 to YOLOv3 (2016 - 2018)

v4



YOLOv4 to YOLOv7 (2020–2021)

v7



Object Tracking Technologies

They work on predicting object trajectory

- DeepSORT (2017):** CNN-Based Appearance Embedding
- ByteTrack (2022):** Associates all detections (high + low confidence)
- BoT-SORT (2023):** Integrates motion and re-ID optimization

Object detector (e.g., YOLO) → feature extraction → data association → trajectory update

Object Tracking Technologies

They work on predicting object trajectory

FairMOT:	Joint Detection and Tracking
CenterTrack:	Regression-Based Tracking
TransTrack:	Transformer-Based Multi-Object Tracking

Jobs

Assign ID, Track Object, Re Assign ID after Disappear

YOLOv8 to YOLOv12 (2020–2025)

Area Attention Mechanism **Block A**

Residual Efficient Layer Aggregation Network (**RELAN**)

Anchor-Free Detection Head

Decoupled Head **Separate Localization and Classification**

Programmable Gradient Information (PGI) **Improve Gradient Flow**

Generalized Efficient Layer Aggregation Network (GELAN)

Focus more on Computation Improvements

YOLOv8 to YOLOv12 (2020–2025)

Layer Nomenclature

C2f: a Cross-Stage Partial (CSP) style block with 2 convolutions; faster than earlier bottlenecks.

C3: CSP block with 3 convolutions (previous versions of YOLO used C3).

A2C2f: Attention-enhanced C2f block (area-attention + C2f) → sometimes also called “R-ELAN variant”.

YOLOv8 to YOLOv12 (2020–2025)

Putting It All Together: YOLOv12 Pipeline

Input Image (e.g., 640×640) → pre-processing (normalisation, etc).

Backbone: stacks of conv blocks (C2f/C3) → intermediate features → attention-enhanced blocks (A2C2f, Area Attention) → deeper features with large receptive field.

Neck: multi-scale feature maps are fused: upsampling/downsampling, concatenation, blocks like A2C2f used for fusion.

Head: For each scale: prediction of bounding boxes (using regression + classification losses) + possibly segmentation/pose.

Post-processing: Non-Max Suppression (or other methods) to filter, plus multi-task outputs if applicable.

Code Walking: YOLO Training

Data Structure

```
DATA_DIR = '/content/data'
```

```
IMAGES_DIR = os.path.join(DATA_DIR, 'images')
```

```
LABELS_DIR = os.path.join(DATA_DIR, 'labels')
```

Train/Test/Split

```
SPLIT_RATIO = (0.7, 0.2, 0.1) # train, val, test
```

Code Walking: YOLO Training

Train/Test/Split

First split: train+val vs test

```
train_val, test_images = train_test_split( valid_images, test_size=test_ratio,  
random_state=42)
```

Second split: train vs val

```
val_size = val_ratio / (train_ratio + val_ratio)  
  
train_images, val_images = train_test_split( train_val, test_size=val_size,  
random_state=42)
```

Code Walking: YOLO Training

Create YAML

```
OUTPUT_DIR = os.path.join(DATA_DIR, 'yolo_dataset')

data_yaml = {
    'path': str(Path(OUTPUT_DIR).absolute()),
    'train': 'train/images',  'val': 'val/images',  'test': 'test/images',
    'nc': len(classes),  'names': classes
}

with open(yaml_path, 'w') as f:
    yaml.dump(data_yaml, f, default_flow_style=False)
```

Code Walking: YOLO Training

Download / Load Model

```
from ultralytics import YOLO  
  
model_v8 = YOLO('yolov8n.pt') # 'n', 's', 'm', 'l', 'x'  
  
# Downloading https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt to 'yolov8n.pt'
```

Modeling

```
results_v8 = model_v8.train(  
    data=str(yaml_path),  
    epochs=EPOCHS,  
    imgsz=IMG_SIZE,
```

Code Walking: YOLO Training

Modeling

patience=50, **Early Stopping Patience**

save=True, **Saves Model Checkpoints (Logs, Weights)**

project=str(**RESULTS_DIR_V8**),

name='candy_detection_v8', **Inside Project this will Appear with Logs, Weights**

exist_ok=True, **Reuse the folder if already Present**

plots=True, **Saves all Line Graphs for training and Validation**

verbose=True **Show Progress in Console**

)

Code Walking: YOLO Training

└─candy_detection_v9

args.yaml

BoxF1_curve.png

BoxPR_curve.png

BoxP_curve.png

BoxR_curve.png

confusion_matrix.csv

confusion_matrix.png

confusion_matrix_normalized.png

labels.jpg

metrics.json

results.csv

results.png

train_batch0.jpg

train_batch1.jpg

train_batch2.jpg

train_batch630.jpg

train_batch631.jpg

train_batch632.jpg

val_batch0_labels.jpg

val_batch0_pred.jpg

val_batch1_labels.jpg

val_batch1_pred.jpg

└─weights

best.onnx

best.pt

best.torchscript

last.pt

Code Walking: YOLO Inference CLI

Image

```
!yolo predict model=/content/runs/detect/train/weights/best.pt source=/content/image.jpg
```

Video

```
!yolo predict model=/content/runs/detect/train/weights/best.pt source=/content/video.mp4  
imgsz=640 conf=0.5 device=0 project=/content/results name=video_pred exist_ok=True
```


Code Walking: YOLO + Tracking

Now you can use Save Yolo file for Detection and one of Tracking Algo with it lets SEE

```
from deep_sort_realtime.deepsort_tracker import DeepSort
```

```
tracker = DeepSort(
```

```
    max_iou_distance=0.7,      Ratio upto Which object is Categorized same
```

```
    max_age=30,      Number of Frames upto which Algo look for Same Object Re-ID
```

```
    n_init=3, Number of consecutive detections required before confirming a New Track
```

```
    nn_budget=100 Memory size for ReID Network
```

```
)
```

Code Walking: YOLO + Tracking

```
model =  
YOLO('/content/v9/weights/best.pt')
```

```
results =  
model.predict(source=frame,  
verbose=False)
```

```
for r in results:  
  
    boxes = r.boxes  
  
    for box in boxes:  
  
        x1, y1, x2, y2 = box.xyxy[0].tolist()  
  
        conf = box.conf[0].item()  
  
        cls = int(box.cls[0].item())  
  
        if cls == pig_class_id and conf > 0.75:  
  
            bbox_xywh = [x1, y1, x2 - x1, y2 - y1]  
  
            detections.append((bbox_xywh, conf, cls))  
  
    tracked_objects = tracker.update_tracks(detections,  
frame=frame)
```

Code Walking: YOLO Detection + Segmentation

...

```
model = YOLO("yolo11n-seg.pt") -seg mean its will give mask as well in Result
```

...

```
results = model(source=..., stream=True) # generator of Results objects
```

```
for r in results:
```

```
    boxes = r.boxes # Boxes object for bbox outputs
```

```
    masks = r.masks # Masks object for segment masks outputs
```

```
    probs = r.probs # Class probabilities for classification outputs
```

Code Walking: YOLO on Videos

```
results = model.predict(  
    source=video_path,  
    save=True,  
    show=False,  
    verbose=False,  
    stream=False  
)
```

This will give BBx, Mask, and Track Object upto Visibility ... Noice

Code Walking: SAM

```
from ultralytics import SAM
```

```
...
```

```
model = SAM("sam2.1_b.pt")
```

```
...
```

```
results = model("/content/image.jpg")
```

```
...
```

Give BBx, Mask, Unique ID to each Object by Default

Code Walking: SAM Video Tracking

As Initial you have to give BBx for Object/s to SAM to Tack

```
from ultralytics.models.sam import SAM2VideoPredictor
```

```
...
```

```
results = yolo_model(first_frame)
```

```
first_frame_boxes = results[0].boxes.xyxy.cpu().numpy()
```

```
...
```

```
sam_predictor = SAM2VideoPredictor(overrides={"model": "sam2.1_b.pt"})
```

```
sam_results = sam_predictor(source=video_path, bboxes=[first_frame_boxes])
```

Code Walking: SAM Video Tracking

```
sam_results =          "frame_id": i,          "class_id": cls_ids[j],          "confidence": confs[j],  
                        "x1": box[0],          "y1": box[1],  
                        "x2": box[2],          "y2": box[3]  
  
....  
  
for i, res in enumerate(sam_results):  
    if hasattr(res, "boxes") and res.boxes is not None:  
        boxes = res.boxes.xyxy.cpu().numpy() # (x1, y1, x2, y2)  
        confs = res.boxes.conf.cpu().numpy() if res.boxes.conf is not None else [None]*len(boxes)  
        cls_ids = res.boxes.cls.cpu().numpy().astype(int) if res.boxes.cls is not None else [-1]*len(boxes)
```

Thanks

Problems

Short-term Occlusion: Trackers lose the object when it goes behind another object or obstacle.

Long-term Reappearance (Re-ID): Reassigning the correct ID after a long disappearance remains difficult.

Life Span: For How long keep track of Trajectory

Fragmentation: When a single continuous object trajectory splits into multiple shorter tracks.

Drift: Gradual misalignment between predicted track and true position due to motion model inaccuracies.